

Simulation Modeling and Analysis

Dr. Md. Nasim Adnan

Random Variables and their Properties

- An *experiment* is a process whose outcome is not known with certainty. The set of all possible outcomes of an experiment is called the *sample space* and is denoted by S .
- The outcomes themselves are called the *sample points* in the sample space.
- If the experiment consists of flipping a coin, then $S = \{H, T\}$
 - where the symbol $\{ \}$ means the “set consisting of,” and “H” and “T” mean that the outcome is a head and a tail, respectively.

- If the experiment consists of tossing a die, then $S = \{1, 2, \dots, 6\}$
 - where the outcome i means that i appeared on the die, $i = 1, 2, \dots, 6$.
 - A random variable is a function (or rule) that assigns a real number (any number greater than $-\infty$ and less than $+\infty$) to each point in the sample space S .

- In general, we denote random variables by capital letters such as X , Y , Z and the values that random variables take on by lowercase letters such as x , y , z .
- The *distribution function* (sometimes called the *cumulative distribution function*) $F(x)$ of the random variable X is defined for each real number x as follows:
- $F(x) = P(X \leq x)$ for $-\infty < x < \infty$

- where $P(X \leq x)$ means the probability associated with the event $\{X \leq x\}$.
- Thus, $F(x)$ is the probability that, when the experiment is done, **the random variable X will take on a value no larger than the number x .**

A distribution function $F(x)$ has the following properties:

1. $0 \leq F(x) \leq 1$ for all x .
2. $F(x)$ is nondecreasing [i.e., if $x_1 < x_2$, then $F(x_1) \leq F(x_2)$].
3. $\lim_{x \rightarrow \infty} F(x) = 1$ and $\lim_{x \rightarrow -\infty} F(x) = 0$ (since X takes on only finite values).

- A random variable X is said to be discrete if it can take at most a countable number of values, such as $x_1, x_2, \dots$ (An example of an uncountable set is all real numbers between 0 and 1.)
- Therefore, a random variable that takes only a finite number of values $x_1, x_2, \dots, x_n$ is a discrete random variable.
- The probability that the discrete random variable X takes on the value x_i is given by:

$$p(x_i) = P(X = x_i) \text{ for } i = 1, 2, \dots$$

- and we must have

$$\sum_{i=1}^{\infty} p(x_i) = 1$$

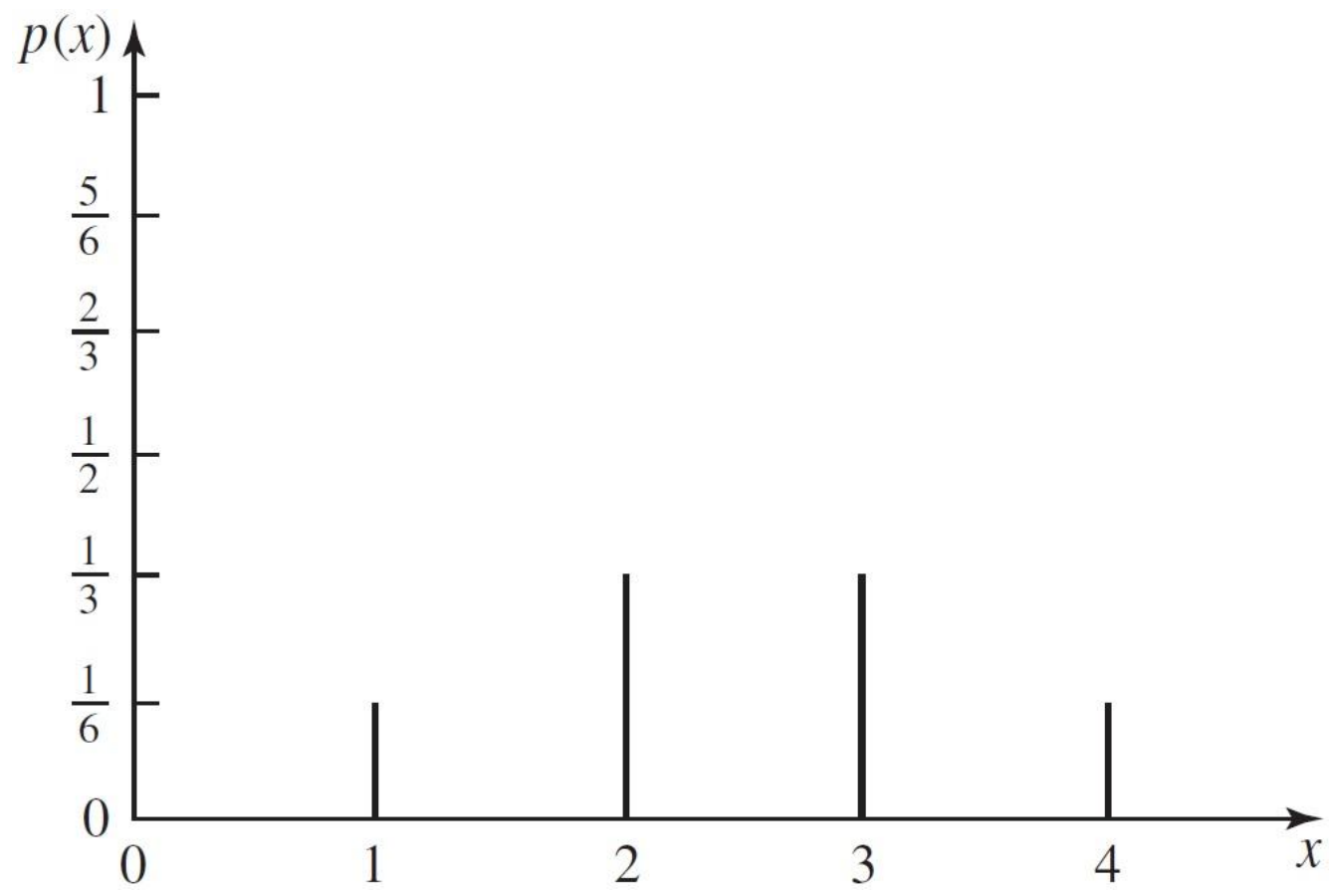
where the summation means add together $p(x_1), p(x_2), \dots$. All probability statements about X can be computed (at least in principle) from $p(x)$, which is called the *probability mass function* for the discrete random variable X . If $I = [a, b]$, where a and b are real numbers such that $a \leq b$, then

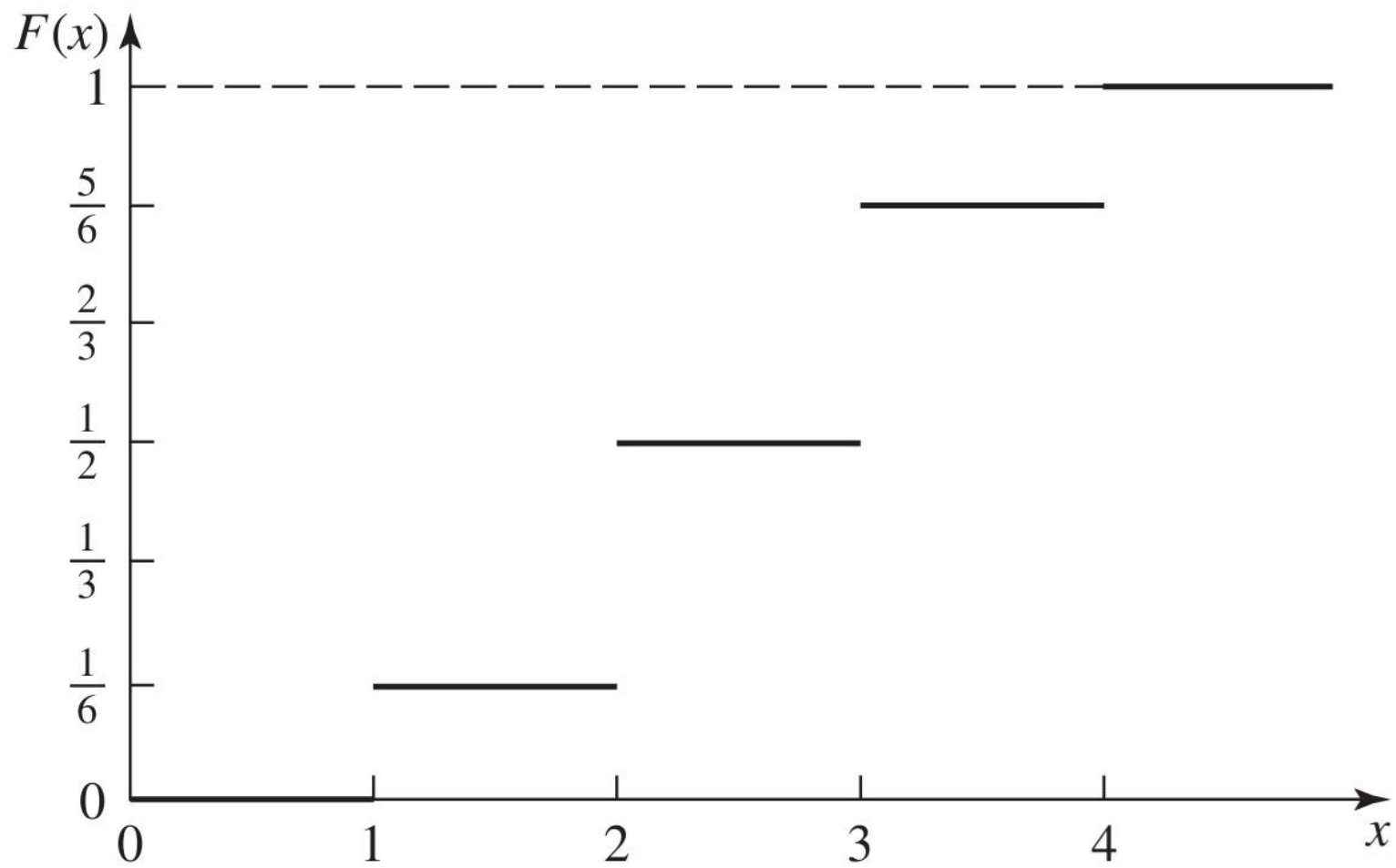
$$P(X \in I) = \sum_{a \leq x_i \leq b} p(x_i)$$

where the symbol $\in$ means “contained in” and the summation means add together $p(x_i)$ for all x_i such that $a \leq x_i \leq b$. The distribution function $F(x)$ for the discrete random variable X is given by

$$F(x) = \sum_{x_i \leq x} p(x_i) \quad \text{for all } -\infty < x < \infty$$

- For the inventory example, the size of the demand for the product is a discrete random variable X that takes on the values 1, 2, 3, 4 with respective probabilities $1/6$, $1/3$, $1/3$, $1/6$.
- The probability mass function and the distribution function for X are given in the following figures.





- We now consider random variables that can take on an uncountably infinite number of different values (e.g., all nonnegative real numbers).
- A random variable X is said to be continuous if there exists a nonnegative function $f(x)$ such that for any set of real numbers B :

$$P(X \in B) = \int_B f(x) \, dx \quad \text{and} \quad \int_{-\infty}^{\infty} f(x) \, dx = 1$$

- Thus, the total area under $f(x)$ is 1.
- Also, if X is a nonnegative random variable, as is often the case in simulation applications, the second range of integration is from 0 to ∞ .
- All probability statements about X can (in principle) be computed from $f(x)$, which is called the probability density function for the continuous random variable X .

For a discrete random variable X , $p(x)$ is the actual probability associated with the value x . However, $f(x)$ is *not* the probability that a continuous random variable X equals x . For any real number x ,

$$P(X = x) = P(X \in [x, x]) = \int_x^x f(y) dy = 0$$

Since the probability associated with each value x is zero, we now give an interpretation to $f(x)$. If x is any number and $\Delta x > 0$, then

$$P(X \in [x, x + \Delta x]) = \int_x^{x + \Delta x} f(y) dy$$

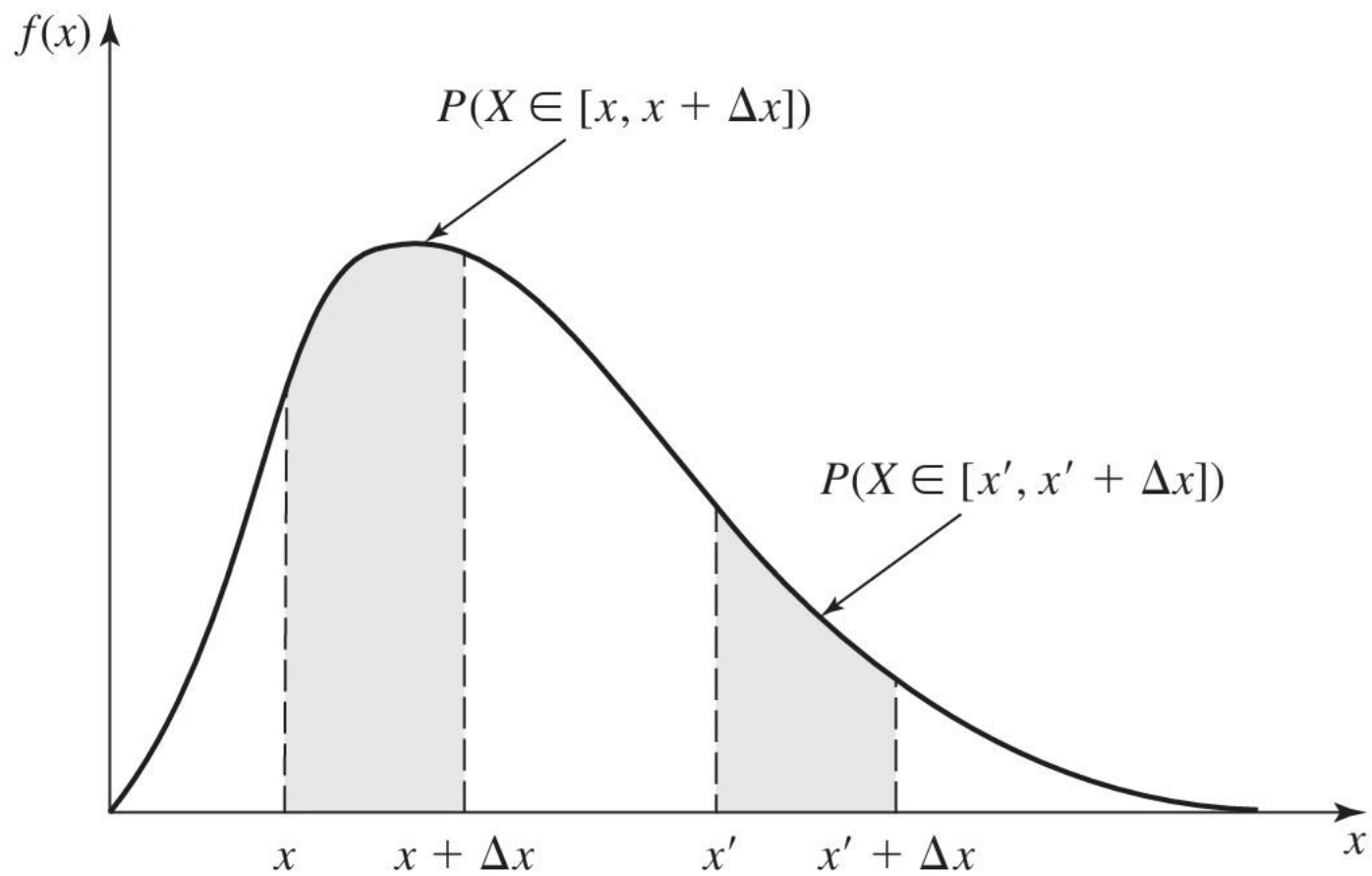
which is the area under $f(x)$ between x and $x + \Delta x$, as shown in Fig. 4.3. It follows that a continuous random variable X is more likely to fall in an interval above which $f(x)$ is “large” than in an interval of the same width above which $f(x)$ is “small.”

The distribution function $F(x)$ for a continuous random variable X is given by

$$F(x) = P(X \in (-\infty, x]) = \int_{-\infty}^x f(y) dy \quad \text{for all } -\infty < x < \infty$$

Thus (under some mild technical assumptions), $f(x) = F'(x)$ [the derivative of $F(x)$]. Furthermore, if $I = [a, b]$ for any real numbers a and b such that $a < b$,

$$P(X \in I) = \int_a^b f(y) dy = F(b) - F(a)$$



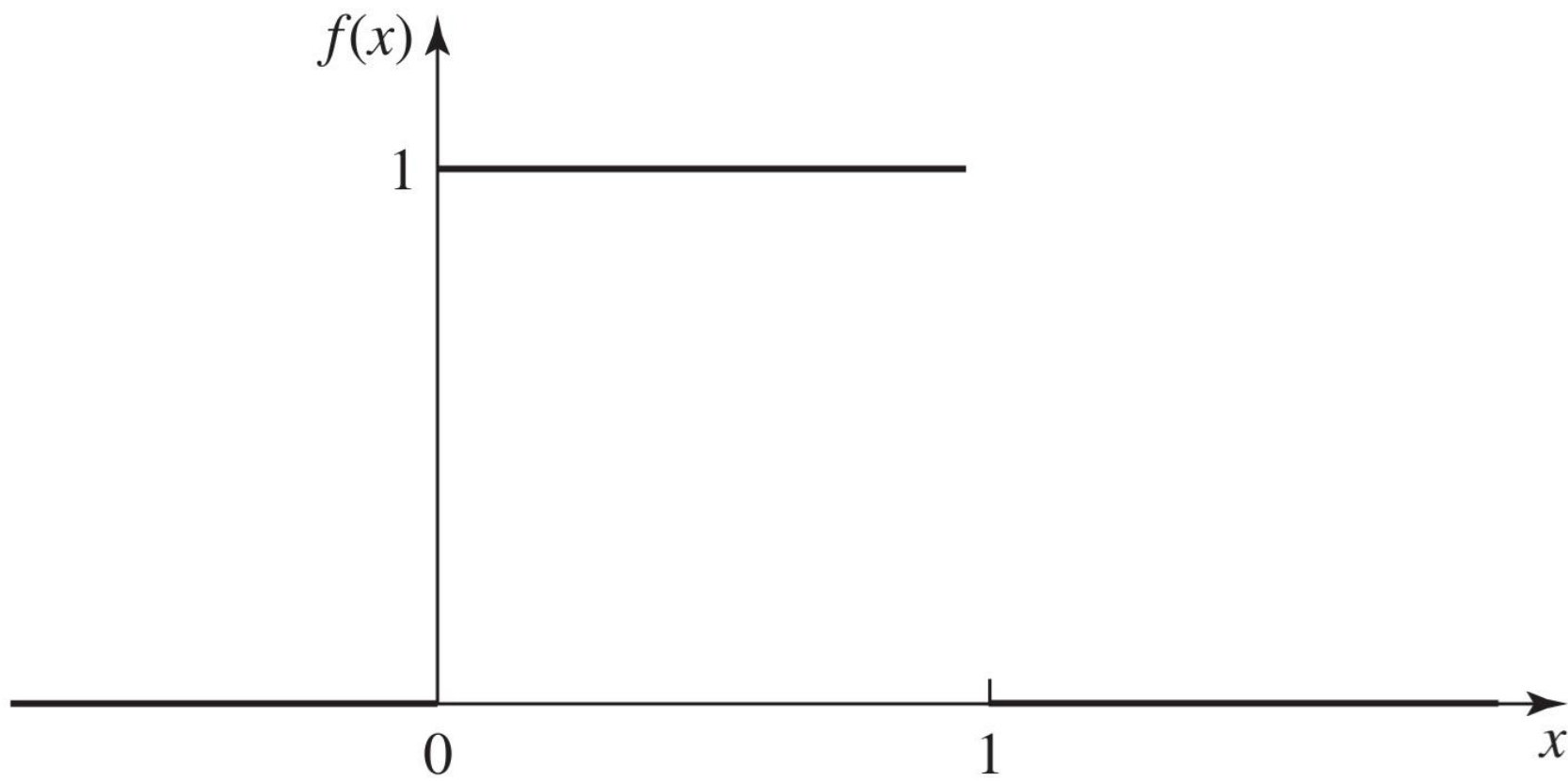
EXAMPLE 4.7. A uniform random variable on the interval $[0, 1]$ has the following probability density function:

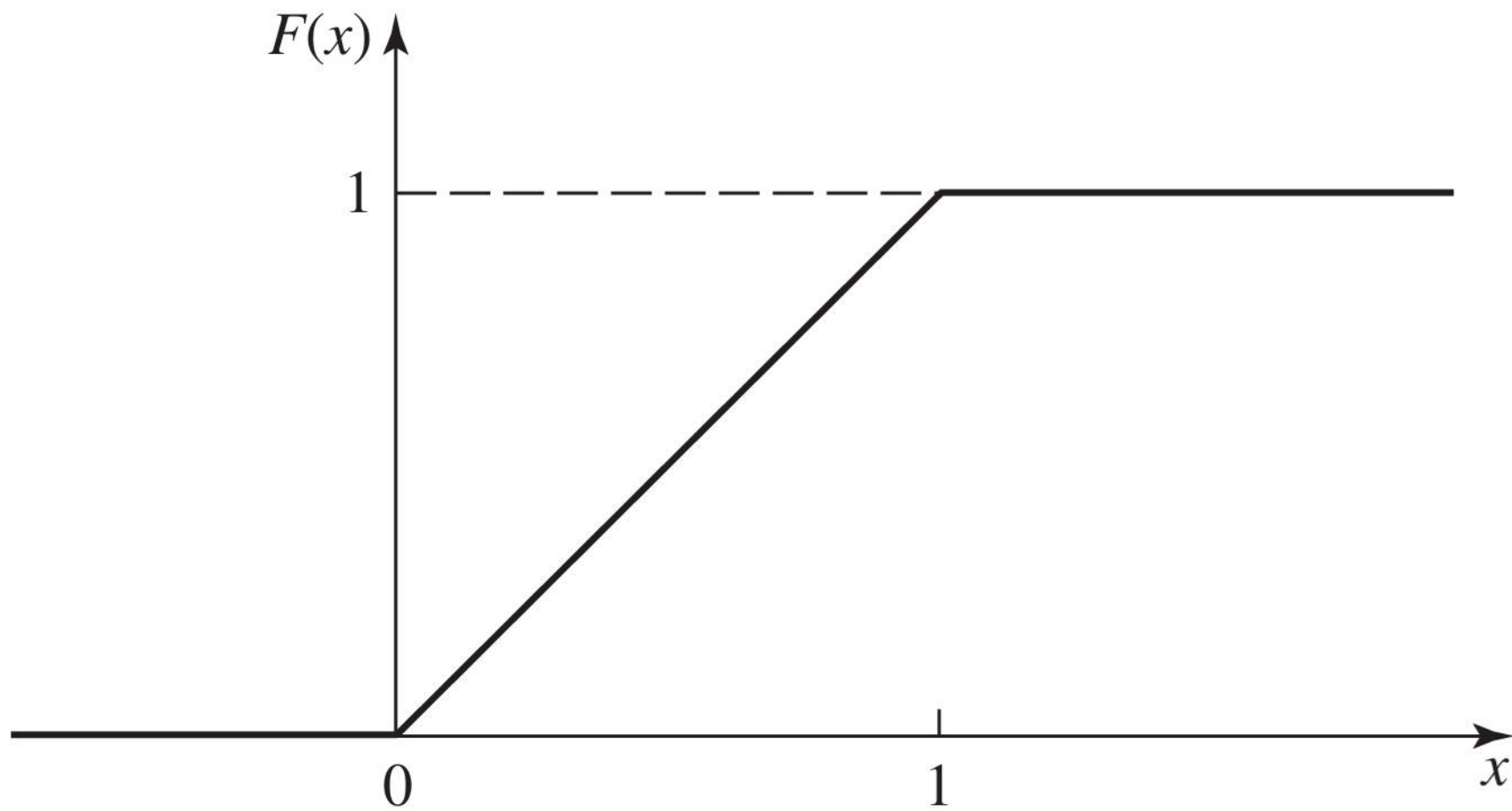
$$f(x) = \begin{cases} 1 & \text{if } 0 \leq x \leq 1 \\ 0 & \text{otherwise} \end{cases}$$

Furthermore, if $0 \leq x \leq 1$, then

$$F(x) = \int_0^x f(y) \, dy = \int_0^x 1 \, dy = x$$

[What is $F(x)$ if $x < 0$ or if $x > 1$?] Plots of $f(x)$ and $F(x)$ are given in Figs. 4.4 and 4.5, respectively.





Finally, if $0 \leq x < x + \Delta x \leq 1$, then

$$\begin{aligned} P(X \in [x, x + \Delta x]) &= \int_x^{x + \Delta x} f(y) dy \\ &= F(x + \Delta x) - F(x) \\ &= (x + \Delta x) - x \\ &= \Delta x \end{aligned}$$

It follows that a uniform random variable on $[0, 1]$ is equally likely to fall in any interval of length Δx between 0 and 1, which justifies the name “uniform.” The uniform random variable on $[0, 1]$ is fundamental to simulation, since it is the basis for generating any random quantity on a computer.

Random-Number Generators

- A simulation of any system or process that has random components (such as a single-server queuing system) needs a way to generate or obtain random numbers.
- We now study some methods of generating random numbers from the uniform distribution on the interval $[0, 1]$; this distribution is denoted by $U(0, 1)$.
- If arithmetic generators are designed carefully, they can produce numbers that behave like independent random values from the $U(0, 1)$ distribution, meaning they pass standard statistical tests for randomness.

- A “good” arithmetic random-number generator should possess several properties:
 - Above all, the generated numbers should appear to be uniformly distributed on $[0, 1]$ and should not be correlated with each other. Otherwise, the simulation results may be completely invalid.
 - From a practical standpoint, the random number generator should also be fast and should not require a large amount of storage.
- We would like to be able to reproduce the same sequence of random numbers exactly for at least two reasons.
 - First, this makes debugging and verifying a computer program easier.

- More importantly, using identical random numbers when simulating different systems helps us make a more accurate and fair comparison between those systems.
 - There should be a way in the generator to easily create separate “streams” of random numbers. These different streams can be thought of as independent generators.
- We would also like the generator to be portable, meaning it should produce the same sequence of random numbers on different computers and with different standard compilers.

LINEAR CONGRUENTIAL GENERATORS

- Many random-number generators in use today are linear congruential generators (LCGs), introduced by Lehmer (1951).
- A sequence of integers $Z_1, Z_2, \dots$ is defined by the recursive formula

$$Z_i = (aZ_{i-1} + c) \pmod{m}$$

- where m (the *modulus*), a (the *multiplier*), c (the *increment*), and Z_0 (the *seed*) are nonnegative integers.

- Therefore, $0 \leq Z_i \leq m - 1$, and to obtain the desired random numbers U_i (for $i = 1, 2, \dots$) in $[0, 1]$, we let $U_i = Z_i/m$.
- We shall concentrate our attention for the most part on the Z_i 's, although the precise nature of the division of Z_i by m should be paid attention to due to differences in the way various computers and compilers handle floating-point arithmetic.
- In addition to non-negativity, the integers m , a , c , and Z_0 should satisfy $0 < m$, $a < m$, $c < m$, and $Z_0 < m$.

- In fact, one can show by mathematical induction that that *every* Z_i is completely determined by m, a, c , and Z_0 .
- However, by careful choice of these four parameters we try to induce behavior in the Z_i 's that makes the corresponding U_i 's *appear* to be *independent and identically distributed* IID $U(0, 1)$ random variables when subjected to a variety of tests.

EXAMPLE 7.2. Consider the LCG defined by $m = 16$, $a = 5$, $c = 3$, and $Z_0 = 7$. Table 7.2 gives Z_i and U_i (to three decimal places) for $i = 1, 2, \dots, 19$. Note that $Z_{17} = Z_1 = 6$, $Z_{18} = Z_2 = 1$, and so on. That is, from $i = 17$ through 32, we shall obtain *exactly* the same values of Z_i (and hence U_i) that we did from $i = 1$ through 16, and in *exactly* the same order. (We do not seriously suggest that anyone use this generator since m is so small; it only illustrates the arithmetic of LCGs.)

The LCG $Z_i = (5Z_{i-1} + 3)(\text{mod } 16)$ with $Z_0 = 7$

i	Z_i	U_i	i	Z_i	U_i	i	Z_i	U_i	i	Z_i	U_i
0	7	—	5	10	0.625	10	9	0.563	15	4	0.250
1	6	0.375	6	5	0.313	11	0	0.000	16	7	0.438
2	1	0.063	7	12	0.750	12	3	0.188	17	6	0.375
3	8	0.500	8	15	0.938	13	2	0.125	18	1	0.063
4	11	0.688	9	14	0.875	14	13	0.813	19	8	0.500

- One objection to LCGs might be that the U_i 's can take on only the rational values $0, 1/m, 2/m, \dots, (m-1)/m$; in fact, the U_i 's might actually take on (only a fraction of) these values, depending on the specification of the constants m, a, c , and Z_0 , as well as on the nature of the floating-point division by m .
- Thus there is no possibility of getting a value of U_i between, say, $0.1/m$ and $0.9/m$, whereas this *should* occur with probability $0.8/m > 0$.
- As we shall see, the modulus m is usually chosen to be very large, say 10^9 or more, so that the points in $[0, 1]$ where the U_i 's can fall are very dense; for $m \geq 10^9$, there are at least a billion possible values.

- Even full-period LCGs, however, can exhibit nonuniform behavior in segments within a cycle.
- For example, if we generate only $m/2$ consecutive Z_i 's, they may leave large gaps in the sequence $0, 1, \dots, m - 1$ of possible values.
- Thus, it is useful to know how to choose m , a , and c so that the corresponding LCG will have full period.
- The following theorem, proved by Hull and Dobell (1962), gives such a characterization:

- The LCG defined previously has full period if and only if the following three conditions hold:
- (a) The only positive integer that (exactly) divides both m and c is 1.
- (b) If q is a prime number (divisible by only itself and 1) that divides m , then q divides $a - 1$.
- (c) If 4 divides m , then 4 divides $a - 1$.

- Obtaining a full (or at least a long) period is just one desirable property for a good LCG.
- We also want good statistical properties (such as apparent independence), computational and storage efficiency, reproducibility, facilities for separate streams, and portability.
- Reproducibility is simple, for we must only remember the initial seed used, Z_0 , and initiate the generator with this value again to obtain the same sequence of U_i 's exactly.
- Also, we can easily resume generating the Z_i 's at any point in the sequence by saving the final Z_i obtained previously and using it as the new seed; this is a common way to obtain nonoverlapping, “independent” sequences of random numbers.

LINEAR CONGRUENTIAL GENERATORS

- Many random-number generators in use today are linear congruential generators (LCGs), introduced by Lehmer (1951).
- A sequence of integers $Z_1, Z_2, \dots$ is defined by the recursive formula

$$Z_i = (aZ_{i-1} + c) \pmod{m}$$

- where m (the *modulus*), a (the *multiplier*), c (the *increment*), and Z_0 (the *seed*) are nonnegative integers.

EXAMPLE 7.2. Consider the LCG defined by $m = 16$, $a = 5$, $c = 3$, and $Z_0 = 7$. Table 7.2 gives Z_i and U_i (to three decimal places) for $i = 1, 2, \dots, 19$. Note that $Z_{17} = Z_1 = 6$, $Z_{18} = Z_2 = 1$, and so on. That is, from $i = 17$ through 32, we shall obtain *exactly* the same values of Z_i (and hence U_i) that we did from $i = 1$ through 16, and in *exactly* the same order. (We do not seriously suggest that anyone use this generator since m is so small; it only illustrates the arithmetic of LCGs.)

The LCG $Z_i = (5Z_{i-1} + 3)(\text{mod } 16)$ with $Z_0 = 7$

i	Z_i	U_i	i	Z_i	U_i	i	Z_i	U_i	i	Z_i	U_i
0	7	—	5	10	0.625	10	9	0.563	15	4	0.250
1	6	0.375	6	5	0.313	11	0	0.000	16	7	0.438
2	1	0.063	7	12	0.750	12	3	0.188	17	6	0.375
3	8	0.500	8	15	0.938	13	2	0.125	18	1	0.063
4	11	0.688	9	14	0.875	14	13	0.813	19	8	0.500

- The “looping” behavior of the LCG in the previous figure is inevitable. By the definition ($Z_i = (aZ_{i-1} + c)(\text{mod } m)$), whenever Z_i takes on a value it has had previously, exactly the same sequence of values is generated, and this cycle repeats itself endlessly.
- The length of a cycle is called the period of a generator. For LCGs, Z_i depends only on the previous integer Z_{i-1} , and since $0 \leq Z_i \leq m - 1$, it is clear that the period is at most m ; if it is in fact m , the LCG is said to have full period.
- Clearly, if a generator is full-period, any choice of the initial seed Z_0 from $\{0, 1, \dots, m - 1\}$ will produce the entire cycle in some order.

EXAMPLE 7.2. Consider the LCG defined by $m = 16$, $a = 5$, $c = 3$, and $Z_0 = 7$. Table 7.2 gives Z_i and U_i (to three decimal places) for $i = 1, 2, \dots, 19$. Note that $Z_{17} = Z_1 = 6$, $Z_{18} = Z_2 = 1$, and so on. That is, from $i = 17$ through 32, we shall obtain *exactly* the same values of Z_i (and hence U_i) that we did from $i = 1$ through 16, and in *exactly* the same order. (We do not seriously suggest that anyone use this generator since m is so small; it only illustrates the arithmetic of LCGs.)

The LCG $Z_i = (5Z_{i-1} + 3)(\text{mod } 16)$ with $Z_0 = 7$

i	Z_i	U_i	i	Z_i	U_i	i	Z_i	U_i	i	Z_i	U_i
0	7	—	5	10	0.625	10	9	0.563	15	4	0.250
1	6	0.375	6	5	0.313	11	0	0.000	16	7	0.438
2	1	0.063	7	12	0.750	12	3	0.188	17	6	0.375
3	8	0.500	8	15	0.938	13	2	0.125	18	1	0.063
4	11	0.688	9	14	0.875	14	13	0.813	19	8	0.500

- Streams are typically set up in a LCG by simply specifying the initial seed for each stream.
- For example, if we want streams of length 1,000,000 each, we set Z_0 for the first stream to some value, then use $Z_{1,000,000}$ as the seed for the second stream, $Z_{2,000,000}$ as the seed for the third stream, and so on.
- Because of condition (a) in the previously demonstrated Theorem, LCGs tend to behave differently for $c > 0$ (called mixed LCGs) than for $c = 0$ (called multiplicative LCGs).

- The LCG defined previously has full period if and only if the following three conditions hold:
- (a) The only positive integer that (exactly) divides both m and c is 1.
- (b) If q is a prime number (divisible by only itself and 1) that divides m , then q divides $a - 1$.
- (c) If 4 divides m , then 4 divides $a - 1$.

Mixed Generators

- For $c > 0$, condition (a) in the Theorem is possible, so we might be able to obtain full period m .
- For a large period and high density of the U_i 's on $[0, 1]$, we want m to be large.
- Furthermore, in the early days of computer simulation when computers were relatively slow, dividing by m to obtain the remainder was a relatively slow arithmetic operation, and it was desirable to avoid having to do this division explicitly...

Multiplicative Generators

- Multiplicative LCGs are advantageous in that the addition of c is not needed, but they cannot have full period since condition (a) of the aforementioned Theorem cannot be satisfied.
- As we shall see, however, it is possible to obtain period $m - 1$ if m and a are chosen carefully.
- It's still computationally efficient to choose $m = 2^b$ and thus avoid explicit division.
- However, it can be shown that in this case the period is at most 2^{b-2} , that is, only one-fourth of the integers 0 through $m - 1$ can be obtained as values for the Z_i 's.

OTHER KINDS OF GENERATORS

- Although LCGs are probably the most widely used and best understood kind of random-number generator, there are many alternative types.
- Most of these other generators have been developed in an attempt to obtain longer periods and better statistical properties.

More General Congruences

- LCGs can be thought of as a special case of generators defined by:
 - $Z_i = g(Z_{i-1}, Z_{i-2}, \dots)(\text{mod } m)$
- where g is a fixed deterministic function of previous Z_j 's. As with LCGs, the Z_i 's defined by the aforementioned Equation lie between 0 and $m - 1$, and the $U(0, 1)$ random numbers are given by $U_i = Z_i/m$.

- One obvious generalization of LCGs would be to let $g(Z_{i-1}, Z_{i-2}, \dots) = a'Z_{i-1}^2 + aZ_{i-1} + c$, which produces a quadratic congruential generator.
- A special case that has received some attention is when $a' = a = 1$, $c = 0$, and m is a power of 2, has better statistical properties.
- Since Z_i still depends only on Z_{i-1} (and not on earlier Z_j 's), and since $0 \leq Z_i \leq m - 1$, the period of quadratic congruential generators is at most m , as for LCGs.

- A different choice of the function g is to maintain linearity but to use earlier Z_j 's; this gives rise to generators called **multiple recursive generators (MRGs)** and defined by
 - $g(Z_{i-1}, Z_{i-2}, \dots) = a_1 Z_{i-1} + a_2 Z_{i-2} + \dots + a_q Z_{i-q}$
- where $a_1, a_2, \dots, a_q$ are constants.
- A period as large as $m^q - 1$ then becomes possible if the parameters are chosen properly.
- L'Ecuyer, Blouin, and Couture (1993) investigated such generators and identified several specific generators of this type that perform well, and give portable implementations.

Composite Generators

- Several researchers have developed methods that combine two or more separate generators to produce the final random numbers.
- The goal of using a composite generator is to achieve a longer period and better statistical properties than those of the individual simple generators.
- However, the main disadvantage of a composite generator is that the cost of generating each U_i is higher compared to using a single simple generator alone.

- One of the earliest types of composite generators used a second LCG to shuffle the output of the first LCG.
- At the beginning, a vector $\mathbf{V} = (V_1, V_2, \dots, V_k)$ is filled one by one with the first k values U_i produced by the first LCG (originally, $k = 128$ was suggested).
- Then, the second LCG is used to generate a random integer i that is uniformly distributed over $1, 2, \dots, k$. The value V_i is returned as the first $U(0, 1)$ variate. After that, the first LCG replaces this i th position in $\mathbf{V}$ with its next U_i value. The second LCG then randomly selects the next returned random number from the updated vector $\mathbf{V}$, and the process continues.
- Shuffling has a strong intuitive appeal because it is expected to break correlations and greatly increase the period of the random-number generator.

- Despite the apparent advantages of rearranging (shuffling) the output of a simple generator, shuffled generators are not as well understood.

For example, with a shuffled generator, it is not possible to jump directly to an arbitrary point in the output sequence without first generating all the intermediate values. In contrast, this is possible with LCGs, which makes them easier to analyze and control.

- Another way to combine two generators (in simplified form) is as follows: Let $\{Z_1 i\}$ and $\{Z_2 i\}$ denote the integer sequences generated by two different LCGs with different moduli. Then define $Z_i = (Z_1 i - Z_2 i) \pmod{m}$ for some integer m , and finally set $U_i = Z_i/m$.
- This idea can be extended to more than two generators and has several advantages. The period is very long (at least 10^{18} in one example), the multipliers in each component generator can be small, which improves portability and allows the generator to run on almost any computer. In addition, the resulting generator is fast, and its statistical properties appear to be very good.

- One very attractive type of random-number generator is created by combining several multiple recursive generators (MRGs) in a specific way.
- In this method, J different MRGs are used, producing the sequences $\{Z_{1,i}\}, \{Z_{2,i}\}, \dots, \{Z_J,i\}$.
- Letting m_1 be the modulus used in the first of these MRGs, and letting $d_1, d_2, \dots, d_J$ be specified constants, we define
 - $Y_i = (d_1 Z_{1,i} + d_2 Z_{2,i} + \dots + d_J Z_{J,i}) \pmod{m_1}$
- and finally return $U_i = Y_i/m_1$ as the i th random number for use in the simulation.

- Although the parameters for these kinds of generators must be chosen very carefully, extremely long periods and exceptionally good statistical properties are possible.
- L'Ecuyer (1999) performed an extensive search to find good parameter settings for these generators. The simplest case, where two MRGs ($J = 2$) are combined, has a very long period of about 2^{191} and also shows excellent statistical properties.

$$Z_{1,i} = (1,403,580Z_{1,i-2} - 810,728Z_{1,i-3})[\text{mod } (2^{32} - 209)]$$

$$Z_{2,i} = (527,612Z_{2,i-1} - 1,370,589Z_{2,i-3})[\text{mod } (2^{32} - 22,853)]$$

$$Y_i = (Z_{1,i} - Z_{2,i})[\text{mod } (2^{32} - 209)]$$

$$U_i = \frac{Y_i}{2^{32} - 209}$$

Feedback Shift Register Generators